Jobnity — Master Technical Reference

Comprehensive Guide to Algorithms, Models, System Connections & End-to-End Execution

Algorithms & Math | System Integration | Data Flow Pipelines | Codebase File Map

1. Algorithms & Mathematical Models

1.1. Length-Sorted Token Boundary Taxonomy Extraction Algorithm

Problem: Standard substring matching causes false positives (e.g., matching "C" inside "C++", "JS" inside "JSON", or "Java" inside "JavaScript").

Algorithm Logic:

1. Load all 1,156 canonical skills and 1,321 skill aliases into a single lowercased dictionary: `lookup_map[term] = canonical_skill_name`.
2. Sort all terms in **descending order of character length** ($|T_1| \ge |T_2| \ge \dots \ge |T_n|$). This guarantees that longer terms like "React Native" or "C++" are matched and processed before shorter terms like "React" or "C".
3. Construct regex patterns using explicit word boundaries:
   - For terms with length > 1: `r'␣' + re.escape(term) + r'␣'`.
   - For single-character terms (e.g., 'R', 'C'): `r'(?<=[\s,.:;/(])' + re.escape(term) + r'(?=[\s,.:;/)]|$)'`.
4. Scan the cleaned text and collect matching canonical names into a set to eliminate duplicate entries.

1.2. Weighted Skill Importance & Gap Calculation Model

Not all skills are equally important. Required core skills carry higher weight than optional preferred skills, and priority levels range from 1 (lowest) to 5 (highest).

```
Skill Weight = Priority * (1.5 if Importance == "Required" else 1.0)

Total Role Weight = ∑ (Skill Weight for all Role Skills)
Earned Skill Weight = ∑ (Skill Weight for Matched Candidate Skills)

Readiness Score (%) = Round( (Earned Skill Weight / Total Role Weight) * 100 )
Skill Gap Score (%) = 100 - Readiness Score
```

1.3. Distinct Skill Guardrail Matrix

To prevent invalid equivalence between related but fundamentally different technologies, the system enforces strict guardrail sets $\mathcal{G}_i$:

```
DISTINCT_GUARDRAILS = [ {"java", "javascript"}, {"docker", "kubernetes"}, {"aws", "azure"},
{"python", "pytorch"}, {"c", "c++"}, {"c++", "c#"}, {"react", "react native"}, {"sql",
"nosql"}, {"html", "css"} ]
```

Rule: If Candidate Skill $S_c \in \mathcal{G}_k$ and Required Skill $S_r \in \mathcal{G}_k$ where $S_c eq S_r$, the engine rejects the match immediately and marks S_r as **Missing**.

2. System Connections & Component Integration

The Jobnity platform connects 5 major system components seamlessly:

Component	Technology	Role & Connection Mechanism
Client UI	React 18 + Vite	Renders user interfaces, manages step navigation state, sends asynchronous REST requests via Axios/Fetch API to backend endpoints.
API Dispatcher	FastAPI + Uvicorn	Asynchronous ASGI web server on port 8001. Handles CORS headers, request validation via Pydantic schemas, and routes requests to Python services.
Storage Engine	OS Tempdir / Local Storage	Detects read-only environments (Vercel Serverless). Dynamic path selection routes DB writes and uploads to `/tmp/jobnity.db` and `/tmp/jobnity_uploads`.
Database Layer	SQLite + SQLAlchemy ORM	Provides relational database access for 52 job roles, 1,156 skills, 1,321 aliases, 125 resources, resumes, candidate profiles, and gap analysis reports.
Cloud Tunnel	Cloudflare Tunnel (`cloudflared`)	Establishes encrypted QUIC connections between local port 8001 and Cloudflare edge servers for `https://jobnity.thundercipher.xyz`.

3. End-to-End Data Execution Pipeline

Step 1: Resume Upload

User drops a PDF/DOCX file into the uploader. Frontend sends `multipart/form-data` to `POST /api/resumes/upload`.

Step 2: File Validation & Saving

Backend validates extension (.pdf/.docx) and size (<10MB). File is written to disk under a unique UUID filename (e.g. `a1b2c3d4.pdf`).

Step 3: Text Parsing & NLP Extraction

`pypdf` or `python-docx` reads raw document text. `NLPExtractorService` splits sections, extracts contact details, education, experience, and runs the taxonomy matcher to extract skills.

Step 4: User Confirmation & Skill Editing

Frontend displays extracted profile. User adds missing skills or removes misidentified items. `PUT /api/resumes/{id}/profile` saves confirmed skills as the single source of truth.

Step 5: Gap Analysis & Roadmap Generation

User selects a target role. `POST /api/analysis/skill-gap` runs the weighted guardrail engine. Missing skills are queried against `LearningResource` table to output course recommendations.

4. Codebase Directory & File Responsibility Map

Directory / File Path	Primary Responsibility
`src/App.jsx`	React Router configuration and page transition animations.
`src/pages/AnalyseResumePage.jsx`	Main stepper flow (Upload → Profile Review → Role Selection → Gap Report).
`src/pages/RoadmapPage.jsx`	Interactive career roadmap view and learning resource cards.
`src/pages/DocsPage.jsx`	Interactive technical documentation viewer inside frontend.
`backend/app/main.py`	FastAPI application initialization, CORS middleware, global exception handler, static asset mounting.
`backend/app/core/config.py`	Serverless environment detection, writable directory resolver, and database path configuration.
`backend/app/api/routes/resumes.py`	Endpoints for uploading resumes, fetching candidate profiles, and updating confirmed skills.
`backend/app/api/routes/analysis.py`	Endpoints for executing skill gap analysis and generating learning roadmaps.
`backend/app/services/resume_parser.py`	PDF extraction (`pypdf`) and Word extraction (`python-docx`).
`backend/app/services/nlp_extractor.py`	Taxonomy alias matcher, section header splitter, and profile entity parser.
`backend/app/services/skill_gap.py`	Weighted importance engine, distinct guardrail enforcement, and readiness score calculator.
`Dockerfile` & `docker-compose.yml`	Multi-stage Node+Python container build and service orchestration.